CSC 583 Natural Language Processing - Fall 2026

Homework #5 Write-Up: Text Summarization & Generation using LLMs

Name: Asad Khan

Collaborators: None. GenAI disclosure: Claude (Anthropic) was used as a coding assistant.

Files: HW5-Part1-AsadKhan.ipynb, HW5-Part2-AsadKhan.ipynb, HW5-Part3-AsadKhan.ipynb (all outputs included).

Environment: all three notebooks were run locally on an Apple M4 laptop (24 GB) with the PyTorch MPS (Metal GPU) backend; torch 2.14, transformers 5.17, datasets 5.0, evaluate 0.4.6. The Colab-only setup lines are kept as comments.

Part I: Summarization using Encoder-Decoder Models

Setup

The test set is drawn from `RealTimeData/bbc_news_alltime`. The notebook lists the dataset's monthly configs and takes the latest one, which is **2025-06** (2,671 articles published 1-27 June 2025). Following the brief, I removed articles with an empty `description` (7). I also removed 3 articles whose `content` was empty, since there is nothing to summarize. I then shuffled with `seed=42` and kept 300 articles. The articles average 575 words, and the reference descriptions average 18 words.

Both models are loaded with `AutoModelForSeq2SeqLM` and run for inference only. Inputs get the `"summarize: "` prefix and are tokenized with `padding=True, truncation=True, max_length=1024`, then `set_format("torch")`. The two tokenizers turned out to produce identical token ids, since Falconsai is a fine-tuned t5-small. 97 of the 300 articles were longer than 1,024 tokens and were truncated.

Both models use the same decoding settings: beam search (4 beams), `no_repeat_ngram_size=3`, and up to 128 new tokens. I first tried a 64-token cap, but many summaries were then cut off mid-sentence. That would have penalized both models for a decoding choice rather than for what they learned. With 128, no summary reached the cap.

Evaluation metrics for the two models

Model (n = 300)	ROUGE-1	ROUGE-2	ROUGE-L	Mean perplexity	Median perplexity	BERTScore P	BERTScore R	BERTScore F1	Mean words
google-t5/t5-small (off-the-shelf)	0.2702	0.1268	0.2221	71.53	51.21	0.8605	0.8849	0.8723	36.6
Falconsai/text_summarization (fine-tuned)	**0.2846**	**0.1372**	**0.2291**	**57.92**	**42.09**	**0.8622**	**0.8966**	**0.8788**	40.9
Reference descriptions (for context)	-	-	-	101.89	59.95	-	-	-	18.1

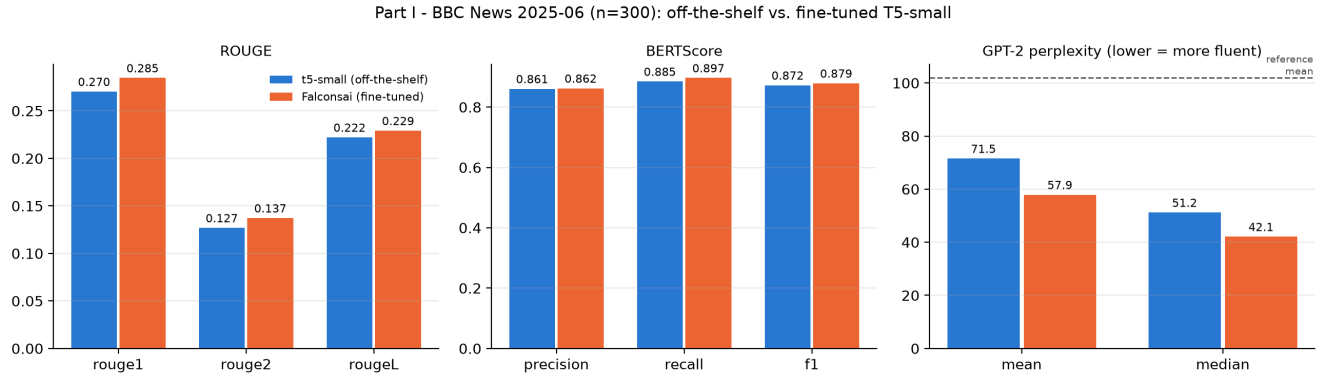


Figure 1. Part I corpus-level metrics. Perplexity is GPT-2's (the evaluate "perplexity" metric is broken on current versions, so I used the replacement function from the brief). The dashed line is the perplexity of the human-written descriptions.

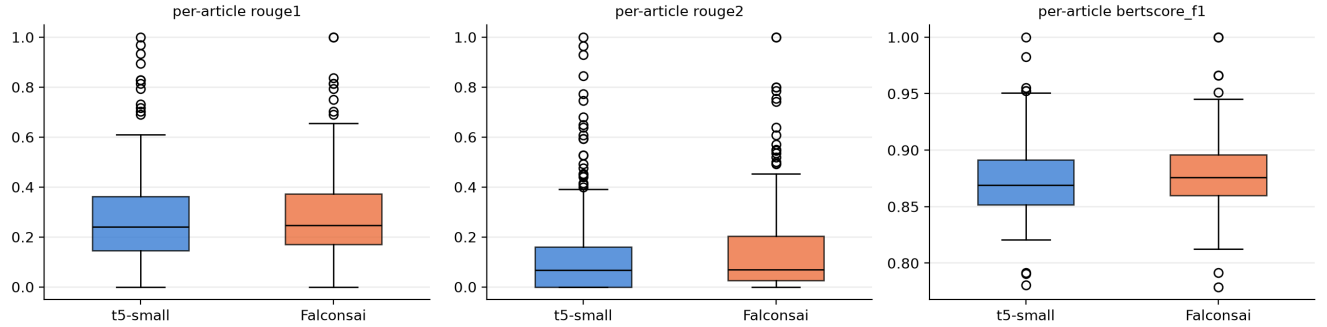


Figure 2. Per-article score distributions. The two boxes overlap almost completely; the fine-tuned model's lower quartile is a little higher.

Comments on the evaluation results

- **Expected vs. observed.** I expected the summarization-tuned model to win clearly. It wins on every metric, but by very little: +0.014 ROUGE-1, +0.010 ROUGE-2, +0.007 BERTScore F1. Per article, Falconsai has the higher ROUGE-1 on 145 articles and t5-small on 133 (22 ties), so this is close to a coin flip. BERTScore F1 favors Falconsai more consistently, 192 to 107.
- **Why the gap is small.** The off-the-shelf "t5-small" is not really a general model for this task. T5 was pre-trained on a multi-task mixture that already included CNN/DailyMail summarization under the same `"summarize: "` prefix. So both models already know how to produce a news summary, and the fine-tuning adds less than the model names suggest. The Falconsai model card describes its fine-tuning data only generally (documents paired with human-written summaries) and does not name a dataset. If that data was news-like, the fine-tuning would mostly reinforce what T5 already does.
- **ROUGE vs. BERTScore.** ROUGE only counts exact word and bigram overlap. ROUGE scores are low for both models because the BBC descriptions are short, rewritten teasers (only 24% of a description's 4-grams appear verbatim in the article). Both models mostly copy sentences: 61% (t5-small) and 70% (Falconsai) of their 4-grams are copied from the article. BERTScore compares contextual embeddings and gives credit for paraphrase. That is why all BERTScore values sit in a narrow 0.86-0.90 band, and why the models look much closer on BERTScore than ROUGE would suggest. Recall is higher than precision for both models: the summaries are about twice as long as the references, so they tend to contain the reference's content plus extra material.
- **Perplexity.** Perplexity looks only at the generated text and measures how predictable GPT-2 finds it. Falconsai's summaries have lower perplexity (57.9 vs. 71.5), partly because they copy more fluent article sentences verbatim. t5-small often lowercases the first word of a sentence ("leah and her husband ...", "a village primary school ..."), which GPT-2 finds surprising. Both models score *below* the human descriptions (101.9). So lower perplexity here means "more ordinary-looking text", not "better summary": the human teasers are deliberately punchy and less predictable.

Two articles: generated summaries

Article A (test index 0): "Stress hormones: Why quick fixes won't lower our cortisol levels" (BBC News, 21 June 2025)	
Article excerpt	"It could apparently change the shape of my face, add pounds to my midriff, and even make my hair fall out. I feel like warnings about cortisol - a stress hormone I know very little about - have hijacked my social media accounts. I see posts advising me to drink a cortisol cocktail ... take a range of different supplements, and massage lavender balm into my temples. ..."
Reference	Can social media hacks really help bring down your cortisol levels?
t5-small (R1 0.086, BERTScore F1 0.847)	warnings about cortisol - a stress hormone I know very little about - have hijacked my social media accounts. I see posts advising me to drink a cocktail, take a range of different supplements, and massage lavender balm into my temples. now, my cheeks seem a touch more puffy than usual and my jeans have started to feel a bit tight.
Falconsai (R1 0.125, BERTScore F1 0.856)	warnings about cortisol - a stress hormone I know very little about - hijack social media accounts. I see posts advising me to drink a cocktail, take a range of different supplements, and massage lavender balm into my temples.

Article B (test index 7): "Pupils told to stay indoors after buzzard nests in playground" (BBC News, 3 June 2025)	
Article excerpt	"To play this video you need to enable JavaScript in your browser. ... The dive-bombing buzzard swoops on an unsuspecting resident. A village primary school in east London has ordered pupils to stay inside due to a dive-bombing buzzard taking up residence in its playground. There are concerns the bird is guarding its nest and may attack children at Dame Tipping Primary School in Havering-atte-Bower ... Havering Council has said its health and safety team had provided 'detailed advice' ..."
Reference	The primary school pupils in Havering are told not to play outside due to the risk from the bird.
t5-small (R1 0.327, BERTScore F1 0.893)	a village primary school in east London has ordered pupils to stay inside. a dive-bombing buzzard swoops on an unsuspecting resident. the buzzard is the most common and widespread bird of prey in the UK.
Falconsai (R1 0.241, BERTScore F1 0.886)	The dive-bombing buzzard swoops on an unsuspecting resident at Dame Tipping Primary School. There are concerns the bird is guarding its nest and may attack children. Havering Council has said its health and safety team has provided advice.

Comments on the generated summaries

- **Both models are extractive.** For both models, nearly every clause above is lifted from the article, which matches the high 4-gram copy rates. Neither model writes an abstract headline-style sentence like the BBC description. Both T5 variants learned summarization from CNN/DailyMail-style data, where the "summary" is a set of highlight sentences close to the lead, so they reproduce the lead.
- **Article A shows the limit of extractive summarization.** The article is a first-person feature, and its first sentences are a personal anecdote. Both models copy that anecdote, including the "I" ("I see posts advising me ..."), which makes no sense in a summary. The reference is a question that frames the article's actual point. Falconsai is shorter and drops the jeans/puffy-cheeks detail. That is marginally better, but it has the same basic problem.
- **Article B: the "general" model wins on ROUGE but not on reading quality.** t5-small's first sentence is essentially the reference, but its third sentence is an off-topic fact about buzzards, and it lowercases sentence starts. Falconsai's summary is more coherent and more specific: it names the school, explains why (the nest), and says what the council did. However, it opens with a photo-caption sentence ("The dive-bombing buzzard swoops on an unsuspecting resident"), a scraping artifact it copied from the page. Neither model filters out boilerplate like this, because they copy rather than understand.
- **Overall:** the fine-tuned model produces slightly cleaner, better-cased, more on-topic summaries. The difference is small because "off-the-shelf" T5 was already multi-task trained on news summarization. On fresh BBC data neither model can do the abstractive reframing the BBC editors do.

Part II: Summarization by Fine-Tuning Encoder-Decoder Models

Setup

Dataset: `knkarthick/samsum` with its own splits (train 14,731 dialogues, as loaded; validation 818; test 819; no empty rows). Dialogues average 94 words and summaries 20 words. Model: `google-t5/t5-small`, the same checkpoint as Part I's off-the-shelf model.

- **Baseline first.** I prepared the test set with `padding=True` and `set_format("torch")` (max 512 tokens). The untouched checkpoint generated summaries with the same decoding as Part I, and I scored them with ROUGE, GPT-2 perplexity and BERTScore. The same functions were later reused, unchanged, for the fine-tuned model.
- **Training data:** a `DatasetDict` with `input_ids`, `attention_mask` and `labels`. Inputs are `"summarize: " + dialogue` truncated at 512 tokens (167 training dialogues were truncated). Labels come from `tokenizer(text_target=summary)`, max 128 tokens. As an optional clean-up step I stripped non-ASCII characters (emoji, curly quotes) and collapsed whitespace.
- **Evaluator during training:** `compute_metrics` decodes the generated ids (replacing -100 with the pad id) and returns ROUGE-1/2/L plus the mean generation length on the validation set. Generation is greedy, with up to 64 new tokens.
- **Training:** `Seq2SeqTrainer` for 3 epochs: batch size 16, linear schedule with 5% warm-up, weight decay 0.01, evaluation and checkpointing every epoch, keeping the best model by validation ROUGE-L. The full run took 53 minutes on the M4's GPU. The fine-tuned model was saved, then reloaded from disk for the test evaluation.

Experimenting with the parameters

The HF summarization guide uses `learning_rate=2e-5`, while T5 is commonly fine-tuned with rates around 1e-4 to 1e-3. Before the full run I compared both on a cheap budget: a fresh t5-small, 1 epoch on 2,000 random training dialogues, scored on 200 validation dialogues.

Learning rate	val loss	val ROUGE-1	val ROUGE-2	val ROUGE-L	mean gen. length (tokens)
2e-5 (HF guide)	2.170	0.3377	0.1167	0.2714	25.4
3e-4	**1.879**	**0.3945**	**0.1625**	**0.3268**	21.0

3e-4 was clearly better after the same number of steps (+0.055 ROUGE-L), so I used it for the full run. With only about 2,760 steps in 3 epochs, 2e-5 moves the weights too little for a 60M-parameter model.

A practical lesson: my first full attempt ran a larger 4-run sweep in the same process and grew to 24 GB of memory, then thrashed swap. With dynamic padding almost every batch has a new shape, and the PyTorch MPS allocator keeps a cached buffer for each one. Padding every batch to a multiple of 64 tokens and releasing the MPS cache every 50 steps kept GPU memory flat at about 4.6 GB.

Training results (3 epochs, validation set)

Epoch	Step	Training loss (last log)	Validation loss	val ROUGE-1	val ROUGE-2	val ROUGE-L	val gen. length
1	921	1.905	1.721	0.4541	0.2176	0.3791	23.0
2	1,842	1.736	1.662	0.4651	0.2240	0.3855	23.4
3	2,763	1.657	**1.655**	**0.4751**	**0.2358**	**0.3959**	23.1



Figure 3. Left: training loss in the learning-rate check. Middle: training and validation loss of the main run. Right: validation ROUGE per epoch. Validation loss is still falling slowly at epoch 3, and training and validation loss are almost equal, so the model is not overfitting yet.

Test-set metrics: original vs. fine-tuned (n = 819)

Metric	Original (off-the-shelf) t5-small	Fine-tuned t5-small	Change
ROUGE-1	0.2732	**0.4647**	+0.1915
ROUGE-2	0.0714	**0.2279**	+0.1565
ROUGE-L	0.2127	**0.3856**	+0.1729
Mean perplexity (GPT-2)	**138.28**	144.12	+5.84
BERTScore precision	0.8568	**0.9209**	+0.0640
BERTScore recall	0.8813	**0.9139**	+0.0325
BERTScore F1	0.8687	**0.9172**	+0.0485
Mean summary length (words)	21.6	17.0	-4.6 (reference: 20.0)

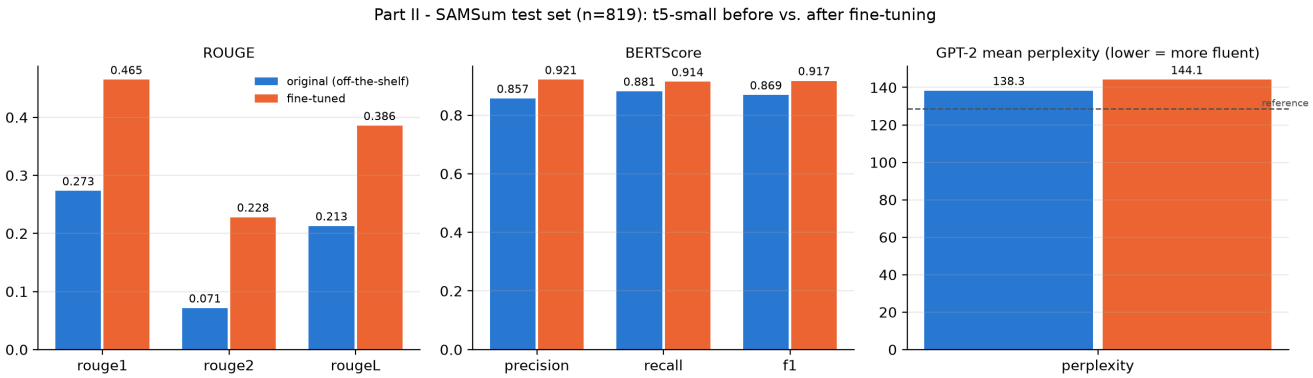


Figure 4. SAMSum test set before vs. after fine-tuning. The dashed line in the perplexity panel is the perplexity of the human reference summaries (128.6).

Comments on the evaluation results

- **Expected vs. observed.** Unlike Part I, here fine-tuning produces a large jump, as I expected: ROUGE-2 more than triples (0.071 to 0.228) and ROUGE-L rises from 0.213 to 0.386. The fine-tuned model has the higher ROUGE-L on 708 of the 819 test dialogues. The reason is the change of domain. T5's built-in "summarize:" skill was learned on news, so on a chat transcript it does what it does on news: it copies "lead" lines. In 55.8% of its outputs the "summary" starts like a chat turn ("Gina: ok, what do u need? ..."), and 82.1% use first or second person. After fine-tuning, 0% start like a chat turn and only 0.2% use first or second person (the references: 0.6%). The model learned SAMSum's house style: third person, names, reported speech.
- **BERTScore** improves less in absolute terms (+0.049 F1), because embedding similarity gives the baseline partial credit for using the right words (the names and topic nouns copied from the dialogue) even when the form is wrong. Precision improves more than recall (+0.064 vs. +0.033): the fine-tuned summaries are shorter and contain less irrelevant material, but they still miss some reference content.
- **Perplexity got slightly worse** (138 to 144), and this is not a sign of worse quality. Perplexity measures how predictable GPT-2 finds the text. Third-person summaries packed with uncommon first names ("Wanda", "Gina") and compressed facts are less predictable than chat lines. The human references themselves have perplexity 128.6, much higher than in Part I. Perplexity is a fluency signal that ignores the source, so it cannot tell a good summary from a fluent, wrong one.

Two test examples: ground truth vs. generated

```
Test index 10:
Wanda: Let's make a party!
Gina: Why?
Wanda: beacuse. I want some fun!
Gina: ok, what do u need?
Wanda: 1st I need too make a list
Gina: noted and then?
Wanda: well, could u take yours father car and go do groceries with me?
Gina: don't know if he'll agree
Wanda: I know, but u can ask :)
Gina: I'll try but theres no promisess
Wanda: I know, u r the best!
Gina: When u wanna go
Wanda: Friday?
Gina: ok, I'll ask
```

Reference	Wanda wants to throw a party. She asks Gina to borrow her father's car and go do groceries together. They set the date for Friday.
Original t5-small (ROUGE-L 0.154)	Gina: ok, what do u need? Wanda: I know, u r the best!
Fine-tuned t5-small (ROUGE-L 0.278)	Gina and Wanda will go to a party on Friday.

```
Test index 50 (offensive language in the original dataset):
Nick: You look absolutely gorgeous and have a lovely smile.
Nick: Would love to get to know you a bit more. How about we meet up for a drink sometime?
Jane: Hmmm... You're shooting a bit above your range aren't you?
Nick: Why would you think that hon?
Jane: Because I'm not that desperate.
Nick: That was a bit below the belt.
Nick: You're nice but you're not THAT hot.
Jane: Oh is your poor little dick shriveling at the thought?
Nick: Actually I'll take it back. Forget about the drink.
Nick: Forget I ever wrote to you.
Jane: Bye loser!
Nick: F***ing b***h!
Jane: You're welcome!
```

Reference	Nick finds Jane pretty and invites her for a drink to get to know her better. Jane rejects Nick and is unpleasant to him. Nick suggests Jane to forget about their conversation.
Original t5-small (ROUGE-L 0.082)	Jane: You're nice but you're not THAT hot. Jane: Actually I'll take it back.
Fine-tuned t5-small (ROUGE-L 0.238)	Nick and Jane are going to meet for a drink.

Comments on the generated summaries (before vs. after fine-tuning)

- **Before fine-tuning, the model does not summarize at all.** It picks two turns and glues them together. In example 50 it even attributes Nick's lines to Jane ("Jane: You're nice but you're not THAT hot."), because it copies the words without tracking who said them.
- **After fine-tuning, the form is right but the meaning is wrong.** Both outputs look exactly like SAMSum summaries: one short third-person sentence that names both participants and states a plan with a time. But both are *unfaithful*. In example 10 Wanda and Gina are not "going to a party on Friday": Wanda wants to *throw* one, and Friday is for grocery shopping, which Gina still has to clear with her father. In example 50 the model says the two "are going to meet for a drink", the opposite of what happened. The model has learned the template "X and Y will/are going to [plan] [time]", which fits most SAMSum dialogues (many are about arranging something). It fills the template with salient words from the dialogue ("party", "Friday", "drink") without understanding refusal, negation or sarcasm.
- **ROUGE rewards this.** The fine-tuned outputs score 2-3 times higher ROUGE-L than the baseline even though one of them reverses the meaning, because they share "Nick", "Jane", "drink" and "Friday" with the reference. A 60M-parameter model trained for 3 epochs gets the style but not the meaning. This motivated looking at faithfulness explicitly in Part III.

Part III: Summarization by Prompting using LLM Decoder Models

LLM used and how it was accessed

I used **OpenAI `gpt-5.4-mini`** through the OpenAI Responses API (Python `openai` SDK) as the summarizer, with reasoning effort set to **`"none"`** so the prompts are compared without hidden "thinking". For the optional LLM-as-a-judge I used a separate, stronger model, **`gpt-5.5`** (reasoning effort "medium"). The key is read from a git-ignored `.env` file, the local equivalent of a Colab Secret, and is never printed. All responses are cached in `part3_llm_cache.json`, so re-running the notebook does not call the API again. The whole part, including the 200-dialogue evaluation and the reasoning comparison, cost well under $1.

The hand-crafted prompts and the reasoning behind them

Each prompt changes one thing relative to the previous one, so differences in the outputs can be traced to the wording.

Prompt 1 - `zero_shot`. The Part II `"summarize: "` prefix, spelled out. It gives no guidance on length or style, so it shows the model's default idea of a "summary".

```
Summarize the following conversation.

{dialogue}
```

Prompt 2 - `constrained` (system role + explicit rules). A persona plus the SAMSum style I observed in the references, written as rules. The length limit addresses the zero-shot verbosity. "Refer to the participants by name" and "third person" match the reference style. "Do not add any detail ... not explicitly in the chat" targets faithfulness. "Notification preview" is a concrete picture of how short the result should be.

```
[system] You are a meticulous note-taker. People paste a messenger chat and you write the gist
that a colleague would read in a notification preview.
[user] Write a summary of the chat below.
Rules:
- One or two short sentences, at most 30 words in total.
- Third person; refer to the participants by their names (never "I", "you", "the user").
- Keep only what matters: the outcome, decision or plan (who will do what, and when, if stated).
- Do not add any detail, motive or emotion that is not explicitly in the chat.
- Plain text only: no bullet points, no quotation marks, no preamble.
Chat:
{dialogue}
```

Prompt 3 - `few_shot` (in-context learning). No rules at all. Three dialogue-to-summary pairs from the SAMSum *training* split (the first three with 30-70 words; never test data) are sent as earlier user/assistant chat turns, followed by the real dialogue. The question is whether the model infers the style from examples alone, the way the fine-tuned T5 had to learn it from data.

```
[user] Summarize the conversation.  Noah: When and where are we meeting? :) ...
[assistant] Noah wants to meet, he quit his job, because his boss was a dick.
[user] Summarize the conversation.  Lucas: Hey! How was your day? ...
[assistant] Demi got promoted. She will celebrate that with Lucas at Death & Co at 10 pm.
[user] Summarize the conversation.  Tina: I'd only like to remind you that you owe me 50 bucks ...
[assistant] Lucy owes Tina 50 dollars. She made a transfer but it is Sunday so ...
[user] Summarize the conversation.  {dialogue}
```

Prompt 4 - `extract_then_write` (structured, grounding first). The model first writes notes (participants, topic, decision/plan with who and when "exactly as stated"), then writes the summary "using ONLY your notes", returned as JSON. The idea is the one Part II exposed: errors come from filling a template with salient words, so forcing the model to state the facts explicitly first should keep the summary grounded.

```
[system] You summarize messenger conversations accurately. You only state facts present in the conversation.
[user] Read the chat below and work in two steps.
Step 1 - notes: list (a) the participants, (b) the main topic, (c) any decision, plan or request,
with who is responsible and when, exactly as stated in the chat.
Step 2 - summary: using ONLY your notes, write a one- or two-sentence third-person summary that
names the participants.
Return JSON with the keys "participants", "topic", "decision_or_plan" and "summary".
Chat:
{dialogue}
```

Outputs for the two Part II test examples

Test index 10 (Wanda and Gina). Reference: "Wanda wants to throw a party. She asks Gina to borrow her father's car and go do groceries together. They set the date for Friday."	
zero_shot (58 words)	Wanda wants to throw a party for fun and asks Gina for help. First, Wanda says she needs to make a list, then asks Gina to see if she can borrow her father's car so they can go grocery shopping together. Gina is unsure her father will agree but promises to ask. They agree to try for Friday.
constrained	Wanda plans a party and needs to make a list first. Gina will ask her father about using his car for Friday grocery shopping with Wanda.
few_shot	Wanda wants to throw a party and is making a list and planning groceries. She asks Gina to borrow her father's car and go shopping on Friday, and Gina says she'll ask him but can't promise.
extract_then_write	Wanda and Gina talked about making a party and planning the preparations. Wanda said she needs to make a list and asked Gina to ask her father about using his car to go grocery shopping together on Friday, and Gina said she would ask but made no promises.
(Part II fine-tuned T5)	Gina and Wanda will go to a party on Friday.

Test index 50 (Nick and Jane). Reference: "Nick finds Jane pretty and invites her for a drink to get to know her better. Jane rejects Nick and is unpleasant to him. Nick suggests Jane to forget about their conversation."	
zero_shot (55 words)	Nick compliments Jane and asks to meet for a drink. Jane dismisses him, insulting him and implying he's out of her league. Nick gets defensive, says she's not that attractive, and tries to withdraw the invitation. The exchange quickly devolves into mutual insults, ending with Jane calling him a loser and Nick responding with profanity.
constrained	Nick asked Jane to meet for a drink, but Jane declined and the exchange ended with both telling each other to forget it.
few_shot	Nick asked Jane out for a drink, but she rejected him and insulted him. They ended up trading insults and angrily ended the conversation.
extract_then_write	Nick and Jane exchange insults after Nick asks Jane to meet for a drink. Nick then says to forget about the drink and to forget that he ever wrote to her.
(Part II fine-tuned T5)	Nick and Jane are going to meet for a drink.

Beyond two examples: the prompts on 200 test dialogues

To check my qualitative impressions I ran every prompt on the first 200 test dialogues and scored them with the Part II metrics, next to both T5 models on the same 200 dialogues.

System (first 200 test dialogues)	ROUGE-1	ROUGE-2	ROUGE-L	Mean perplexity	BERTScore P	BERTScore R	BERTScore F1	Mean words
T5 original (Part II)	0.2571	0.0604	0.2006	148.7	0.8541	0.8794	0.8664	21.3
T5 fine-tuned (Part II)	**0.4515**	**0.2155**	**0.3758**	156.5	**0.9208**	0.9127	**0.9165**	16.4
gpt-5.4-mini / zero_shot	0.3157	0.1068	0.2393	58.4	0.8678	0.9185	0.8923	61.0
gpt-5.4-mini / constrained	0.4492	0.1815	0.3523	146.8	0.9099	0.9190	0.9143	20.0
gpt-5.4-mini / few_shot	0.4119	0.1563	0.3166	86.1	0.8935	**0.9243**	0.9085	33.5
gpt-5.4-mini / extract_then_write	0.3872	0.1486	0.3028	55.3	0.8917	0.9230	0.9069	36.5
Human reference	-	-	-	134.6	-	-	-	19.6

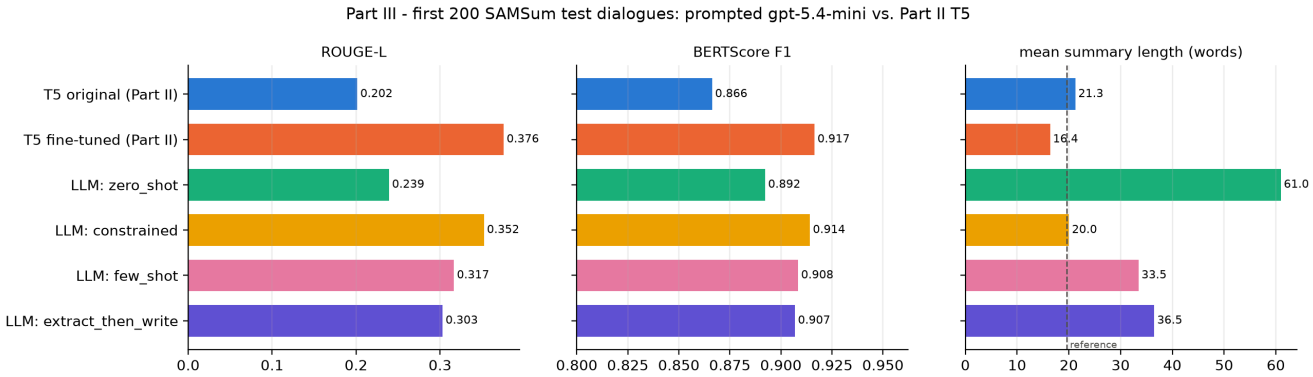


Figure 5. Prompted gpt-5.4-mini vs. the Part II T5 models on the same 200 test dialogues. Length (right) explains most of the ROUGE differences between prompts.

Comments on the generated summaries

- **The LLM gets the meaning right where the fine-tuned T5 did not.** On both examples, every prompt states the essential facts correctly: Wanda is *throwing* a party, Friday is for groceries, and Gina only promises to *ask* about the car; Jane *rejects* Nick. These are the facts the fine-tuned T5 got wrong. The LLM handles negation, refusal and the conditional "I'll try but no promises" because it has general language understanding from large-scale pre-training and instruction tuning, not just the SAMSum template.
- **Zero-shot is faithful but far too long.** With no length guidance the model writes a retelling of the dialogue (61 words on average, 3 times the reference). Its BERTScore recall (0.919) is high because everything is in there, but precision (0.868) and ROUGE-L (0.239) are low. It also has the lowest perplexity (58): long, fluent, generic prose is exactly what GPT-2 predicts well. This shows again that perplexity measures fluency, not summary quality.
- **Constrained is the closest match to the references.** The explicit rules cut the length to 20.0 words (reference: 19.6) and raise ROUGE-L from 0.239 to 0.352, within 0.024 of the fine-tuned T5, with no training at all. The two example summaries are 26 and 23 words. The cost of compression shows in example 50: "both telling each other to forget it" is a small inaccuracy (only Nick said it), and the insults are dropped. The "notification preview" framing made the model compress harder than the references do.
- **Few-shot learns the style partly.** Three examples made the output third person and more SAMSum-like ("Gina says she'll ask him but can't promise"). Without an explicit length limit, though, it still averages 33.5 words. Its summary for example 50 is the best of all the systems: concise, accurate, and it captures both the rejection and the insults. In-context examples convey tone and form well but not a precise length target.
- **Extract-then-write is the most faithful but the least summary-like.** Its notes (see notebook, Section 3) are accurate and complete, but the summary written from them is detailed and procedural ("Wanda said ... and asked Gina to ask ..."), averaging 36.5 words. Grounding the summary in notes increased detail rather than reducing errors. On SAMSum the LLM was already faithful, so the extra structure mainly added length.
- **Fine-tuned T5 still "wins" on ROUGE and BERTScore** (ROUGE-L 0.376 vs. 0.352 for the best prompt), even though on the two examples I would rank every LLM summary above it. The 60M-parameter T5 was trained on 14.7k SAMSum summaries and reproduces their vocabulary and phrasing. The LLM paraphrases (e.g. "grocery shopping" vs. "do groceries"), which ROUGE penalizes. Reference-overlap metrics measure similarity to *this* annotator's wording, not correctness.

(Optional) A reasoning setting: does extra "thinking" help?

constrained prompt, 200 dialogues	ROUGE-L	BERTScore F1	Mean words	Mean output tokens	of which reasoning
reasoning effort "none"	0.3523	0.9143	20.0	28.2	0
reasoning effort "high"	0.3611	0.9166	18.7	236.4	207.6

High reasoning effort gives a small gain (+0.009 ROUGE-L, +0.002 BERTScore F1) for **8.4 times the output tokens**, so it is mostly cost. It did fix example 50: "Jane rejected him and the chat ended with them trading insults" replaces the slightly wrong "both telling each other to forget it". So the thinking helps faithfulness on the occasional tricky dialogue. For a short, easy task like chat summarization, though, it is not worth the extra tokens.

(Optional) LLM-as-a-judge

`gpt-5.5` saw the dialogue, the reference, and all seven candidate summaries under shuffled anonymous letters. It scored each summary 1-5 for faithfulness, coverage and coherence, with a one-sentence justification. The reference itself was included as a sanity check.

System	Test 10 (Wanda/Gina)			Test 50 (Nick/Jane)		
	Faithful.	Coverage	Coherence	Faithful.	Coverage	Coherence
Reference (human)	5	4	5	5	4	5
T5 original	5	1	1	1	1	2
T5 fine-tuned	2	1	4	1	1	5
LLM zero_shot	5	5	5	5	5	5
LLM constrained	5	4	5	3	3	4
LLM few_shot	5	5	5	5	4	5
LLM extract_then_write	5	5	5	5	4	5

- **Where the judge agrees with me.** It catches exactly the errors I found. Fine-tuned T5 gets faithfulness 2 and 1 ("incorrectly says they will go to a party on Friday"; "directly contradicts the dialogue because Jane rejects the drink invitation") while getting coherence 4-5, which separates "reads well" from "is true" just as my analysis did. It also penalized the constrained prompt on example 50 for "incorrectly says both told each other to forget it".
- **Where I disagree.** The judge gave the 60-word zero-shot output perfect scores on both examples. Its rubric rewards coverage, and nothing in it penalizes length, even though that output is too long to be a useful summary. A human reader (or the SAMSum annotators) would prefer the few-shot version. It also gave the original T5's quoted lines faithfulness 5 on example 10, technically true because the words are copied, but misleading for a "summary" that summarizes nothing. An LLM judge is a useful second opinion that catches factual errors ROUGE misses, but its verdicts depend heavily on the rubric I give it.

General Reflections

- **Difficult parts and how I resolved them.** (1) *Memory on the Mac GPU.* My first Part II run grew to 24 GB and swapped for two hours without finishing the first epoch. I traced it to the MPS allocator caching a buffer for every distinct batch shape. Padding batches to multiples of 64, freeing the cache in a `TrainerCallback`, and making the learning-rate sweep cheaper fixed it (flat at about 4.6 GB, 53-minute run). (2) *Library version changes.* transformers 5 replaced `warmup_ratio` with a fractional `warmup_steps`, and datasets 5 returns lazy `Column` objects (no `.shape`; I had to convert them with `list(...)`). (3) *The perplexity metric* is broken, as the brief warns, so I used the provided GPT-2 replacement. (4) *Decoding length:* a 64-token cap cut Part I summaries mid-sentence, so I raised it to 128 for every model. (5) *Data quality:* some BBC articles had empty content or caption boilerplate, which I filtered or discuss above.
- **What I learned.** Whether fine-tuning helps depends on how far the target domain is from what the model already knows. On news (Part I) the extra fine-tuning barely mattered, because T5 had already been trained on news summarization. On chat dialogues (Part II) it transformed the output, but mostly its *form*: the fine-tuned model learned SAMSum's style while still getting facts wrong. I also learned that each metric captures one aspect only: ROUGE measures wording overlap, BERTScore semantic overlap, and perplexity fluency. None of them measures faithfulness to the source, and the fine-tuned T5 scored best on ROUGE while contradicting the dialogue. Prompting a modern LLM gave the most faithful summaries with no training, and short, explicit length and style rules mattered more than clever structure.
- **Overall difficulty.** Moderate to hard. The NLP concepts were manageable, but the engineering took the most time: library versions, long training runs, and getting a fair, identical evaluation pipeline for every model.
- **What I would do differently.** I would test memory and speed on a short run before launching a long one (a 5-minute check would have saved the first two hours). I would also add a faithfulness measure to the automatic evaluation from the start (e.g. an NLI-based check, or the LLM judge run on all test dialogues rather than two), and calibrate the judge's rubric to penalize unnecessary length. With more time I would fine-tune `flan-t5-base` for comparison, to see whether a larger fine-tuned model closes the faithfulness gap with the LLM.
- **Anything else.** Example 50 in SAMSum contains offensive language; it is reproduced (partly masked) only because the starter code fixes these test indices. All metric tables in this write-up are produced by the notebooks and saved as CSV files next to them.